

ENI-ABT

*Administration des Bases de données :
INTRODUCTION*

Module 1

Objectifs d'apprentissage

- Donner un exemple d'interrogation d'un SGBD (et donc d'une base de données) en utilisant un langage de programmation L3G comme java.
- Comparer les SGBDS sur le marché et énumérer les critères de comparaison.
- Définir la notion de datawarehouse.
- Décrire une alternative aux bases de données relationnelles, les bases de données objets, et dresser une grille de comparaison sommaire entre les deux approches

Plan

- Interface entre un L3G et SGBD (java et Oracle)
- SGBD sur le marché
- DataWarehouse
- Base de données Objet
- Fin cours

Introduction

- Il est possible d'exécution des commandes SQL à partir d'un programme écrit en un langage de 3eme génération (java par exemple)
- Et aussi exécuter du code PL/SQL dans du code java
- Mode de fonctionnement:
 - Établir une connexion avec la BD
 - Envoyer les instructions SQL
 - Traiter le résultat

Java + SQL

- Package `java.sql`: Accès et traitement des données dans une base de données
- `java.sql.DriverManager`: gérer des sources JDBC
- `java.sql.Connection`: Spécifier la connexion à la BD
- `java.sql.Statement`: Exécuter et retourner le résultat SQL

Java + SQL

```
public class baseDonnees {  
    .....  
    Class.forName("oracle.jdbc.driver.OracleDriver"); //initialisation de la  
    connexion.  
    DriverManager.setLogStream(System.out);  
    Connection con = DriverManager.getConnection  
    ("jdbc:oracle:thin:@ora8i.server.usegoul.mù:1521:ora8i", login,  
    passwd);  
    Statement stmt = con.createStatement();  
    .....  
}
```

Java + SQL

```
public class baseDonnees {  
    .....  
    Statement stmt = con.createStatement();  
    stmt.executeUpdate(requête sql); //soit Insert, Update or Delete  
  
    ResultSet rs =stmt.executeQuery (requête sql); //les requêtes  
    SQL DML  
    .....  
}
```

Java + SQL

```
ResultSet rs = stmt.executeQuery("select * from client");  
    //Définition de la requête  
ResultSetMetaData rsmd = rs.getMetaData();  
int numCols = rsmd.getColumnCount(); //Le nombre de colonnes  
boolean more = rs.next();  
//Parcours des colonnes pour afficher leur labels ainsi que les  
    valeurs et ceci pour chaque enregistrement  
while (more) {  
    for (i=1; i<=numCols; i++)  
    {  
        rsmd.getColumnLabel(i);  
        rs.getString(i);  
    }  
    more = rs.next(); //passage au prochain enregistrement  
}
```

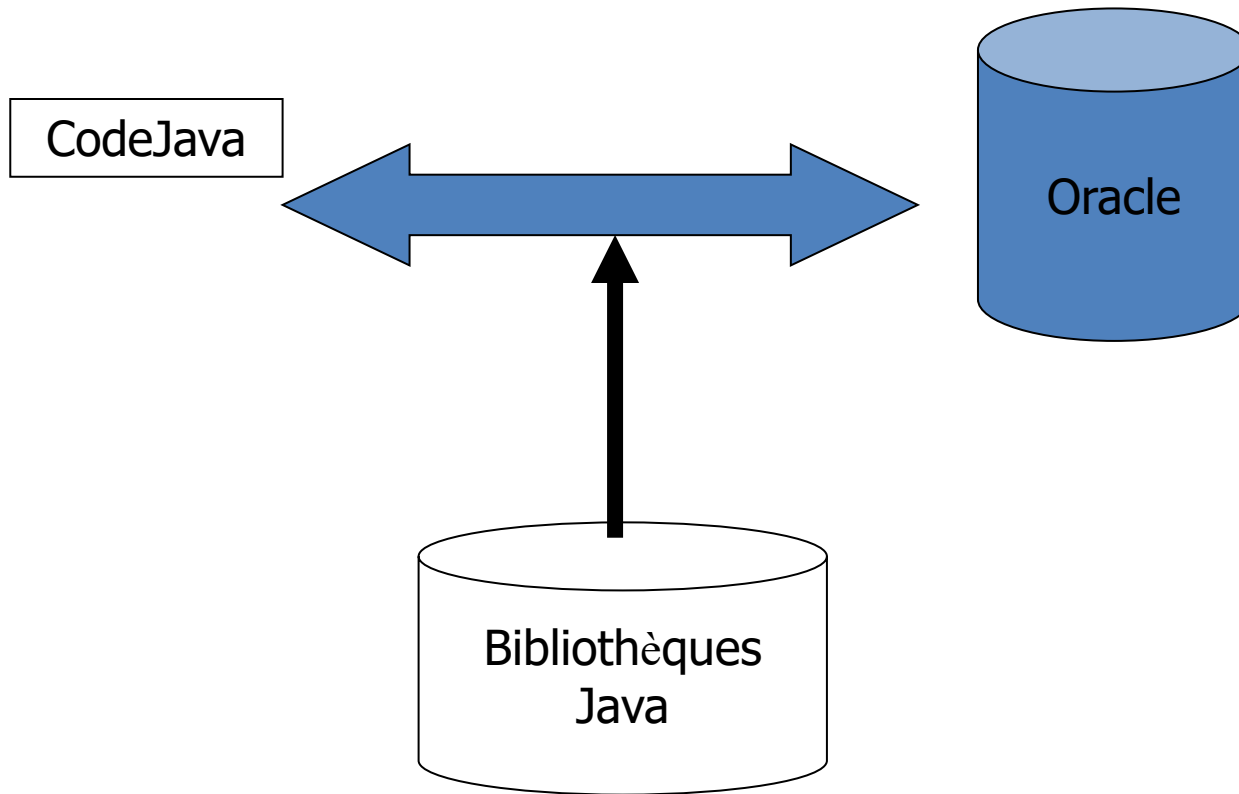

Java + SQL

- Toutes les commandes de BD sont entre un try-catch (Exception e)
- `java.sql.ResultSet`: le résultat d'une requête
- `i` = le numéro de la colonne
- `next ( )` : avancer dans les tuples

Java +PL/SQL

- PL/SQL
 - PL/SQL est destiné aux développeurs Oracle
 - PL/SQL est non portable à d'autres SGBDs
 - Il est plus facile à manipuler que Java (proche de la BD)
- Java
 - Java est non propriétaire à Oracle
 - Java offre des conversions de données SQL ou PL/SQL avec des types Java
 - Plus difficile que PL/SQL (création de classes, héritage, agrégation, etc)
 - Permet une ouverture de la BD à d'autres applications

Java + PL/SQL



Java + PL/SQL

- `java.sql.Statement` : Exécuter du SQL et récupérer le résultat de l'exécution
- `java.sql.CallableStatement` : Exécution de procédures SQL stockées, donc du code PL/SQL

```
stmt.execute ("create or replace function RAISESAL (name CHAR,  
raise NUMBER)return NUMBER is begin return raise + 100000;  
end;");
```

```
CallableStatement cstmt = conn.prepareCall("{? = call RAISESAL (?,  
?)}"); //Préparer l'appel a la fonction
```

Java + PL/SQL

`cstmt.registerOutParameter (1, Types.INTEGER);`//identification de l'index de la valeur de retour ainsi que son type

`cstmt.setString (2, nomPersonne);`//identification du 1er paramètre

`cstmt.setInt (3, salaire);`//identification du 2eme paramètre

`ResultSet rs1 = cstmt.executeQuery();`//exécution de la procédure

`nom1.setText(Integer.toString(cstmt.getInt(1)));`//récupérer le résultat de la fonction

Plan

- Interface entre un L3G et SGBD (java et Oracle)
- SGBD sur le marché
- DataWarehouse
- Base de données Objet
- Fin cours

Les SGBD sur le marché

- Oracle : Oracle
- SQL Server : Microsoft
- DB2 : IBM
- Informix : IBM
- Ingres: Computer Associates
- Adaptative Server Entreprise: Sybase
- Les trois premiers sont le plus présents sur le marché

Les SGBD sur le marché

- Paramètres de comparaison: application et technologie
- Application:
 - traitements intelligents (Business Intelligence): extraction d'information
 - applications complexes (Complex Applications): tels que gestion financière
 - Types de données (Content Services): musique, xml, html, etc...

Les SGBD sur le marché

- Application:
 - E-commerce
 - M-commerce: applications mobiles
 - Les métadonnées (Metadata Services): données sur les données
 - XML-services

Les SGBD sur le marché

- Technologie:
 - Fonctionnalités offertes (Availability): telles que de reprise
 - Facilité d'utilisation (Ease of Use): les Wizards
 - Interopérabilité (Interoperability): ouverture à d'autres systèmes
 - Sécurité (Security): sécurité d'accès à l'information

Les SGBD sur le marché

Application Ratings

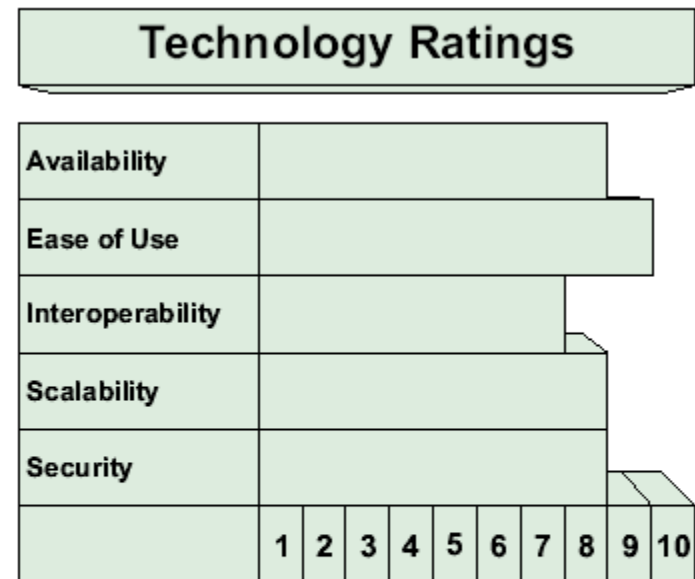
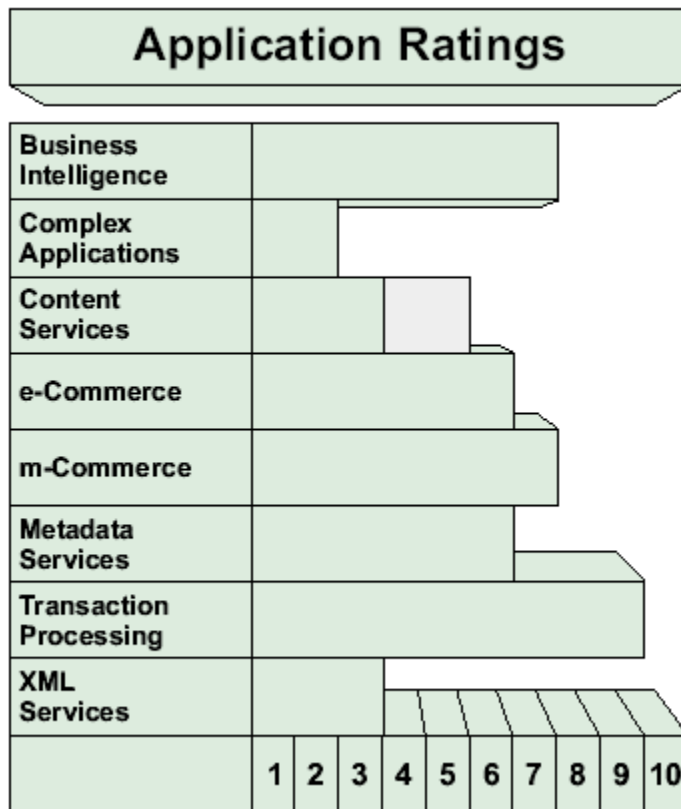
Business Intelligence										
Complex Applications										
Content Services										
e-Commerce										
m-Commerce										
Metadata Services										
Transaction Processing										
XML Services										
	1	2	3	4	5	6	7	8	9	10

Technology Ratings

Availability										
Ease of Use										
Interoperability										
Scalability										
Security										
	1	2	3	4	5	6	7	8	9	10

IBM DB2 UDB v7.2

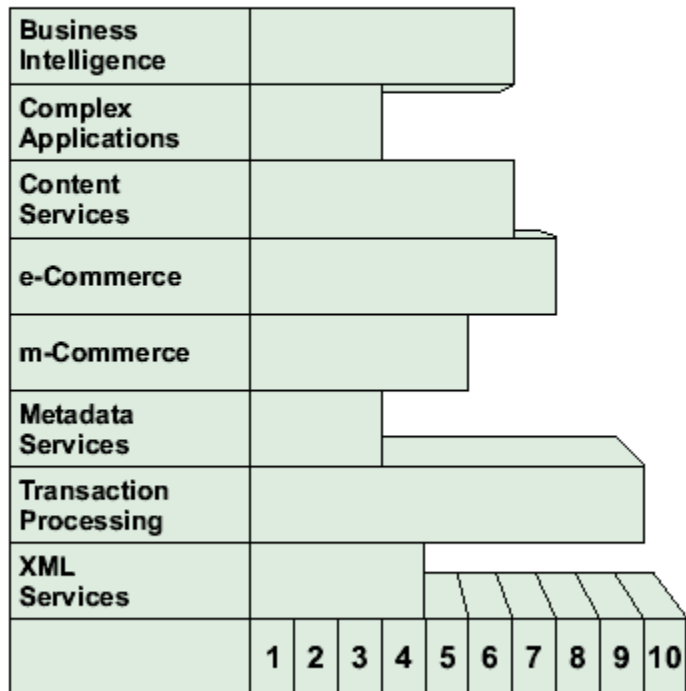
Les SGBD sur le marché



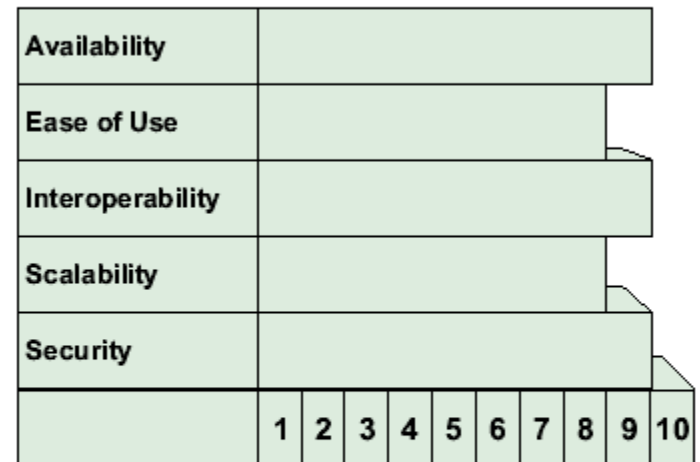
Microsoft SQL Server 2000

Les SGBD sur le marché

Application Ratings



Technology Ratings



Oracle 9i

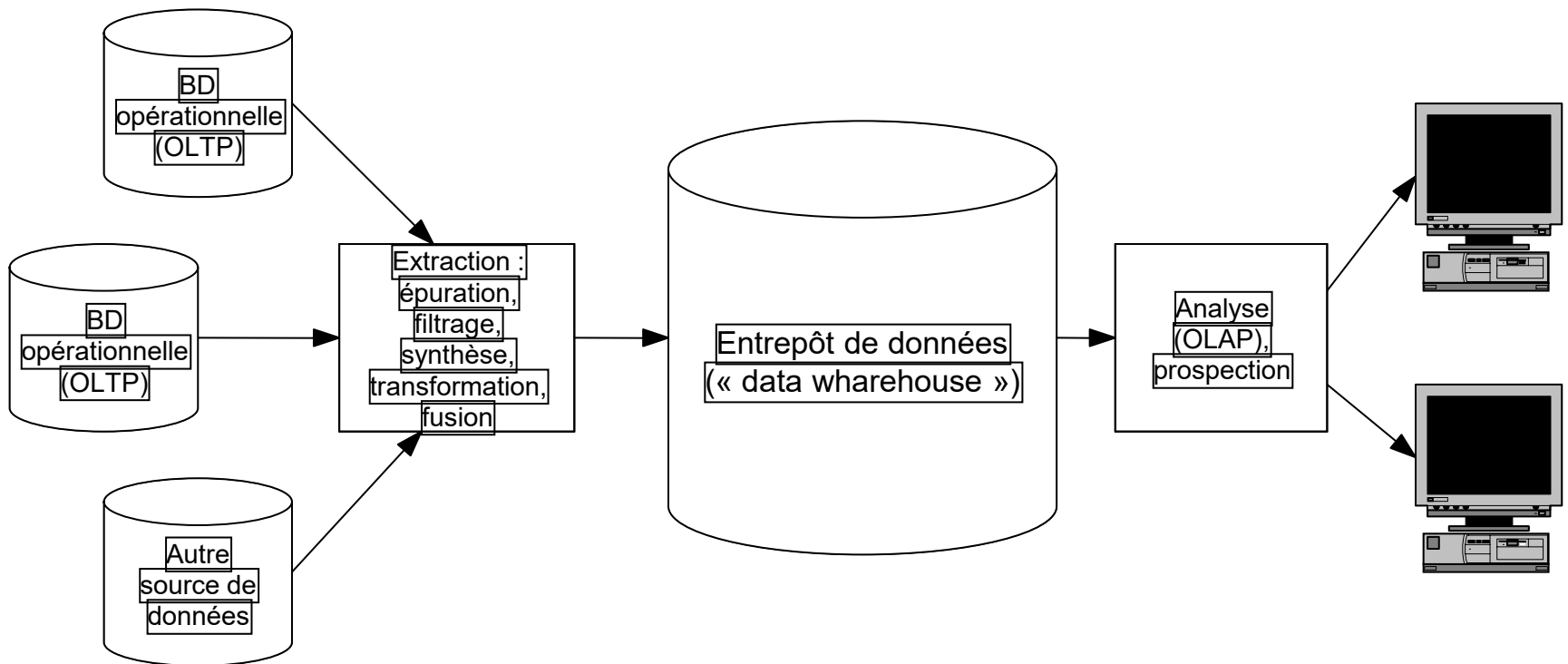
Plan

- Interface entre un L3G et SGBD (java et Oracle)
- SGBD sur le marché
- DataWarehouse
- Base de données Objet
- Fin cours

DataWarehouse

- SGBD limités à traiter plusieurs opérations de jointure
- Proposer de nouveaux modèles qui répondent aux besoins des décideurs
- Solution: dénormalisée le modèle relationnel pour accélérer la recherche d'informations
- Les données peuvent provenir de plusieurs sources (exemple: finance, comptabilité, ...)
- Il faut définir des mécanismes de fusion et de filtrage de données.

DataWarehouse

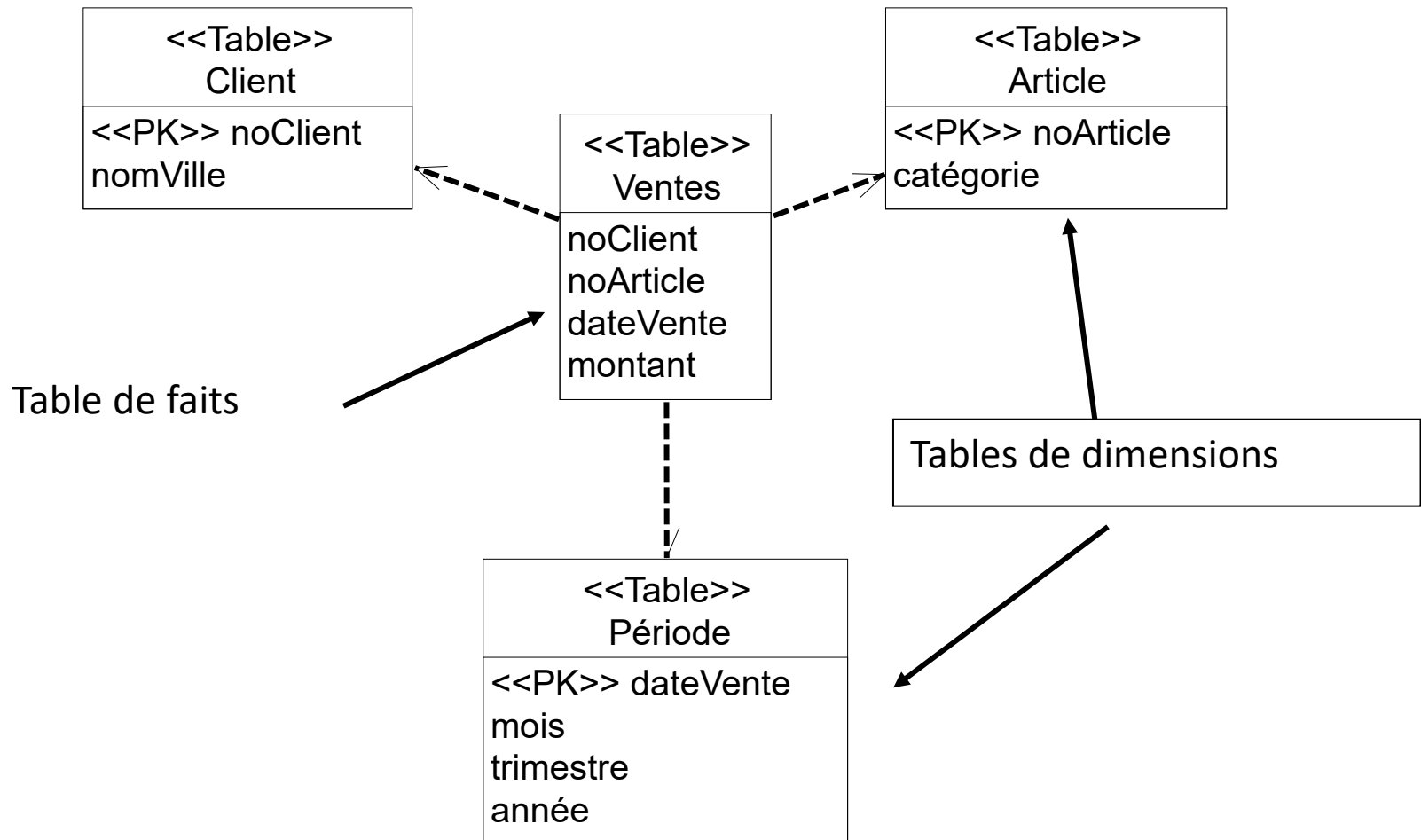


Chaîne de traitement : extraction, transformations, analyses

DataWarehouse

- Les informations recherchées peuvent porter sur différentes dimensions
- Par exemple: la moyenne des ventes par client, par article et par période de temps

DataWarehouse



DataWarehouse

- Les entrepôts de données sont d'une grande taille
- Besoin d'entrepôts de données spécifiques: un pour finance, un pour comptabilité, etc
- Ce sont les dataMart.
- Un dataMart: une vue particulière de l'application pour un décideur

Plan

- Interface entre un L3G et SGBD (java et Oracle)
- SGBD sur le marché
- DataWarehouse
- Base de données Objet
- Fin cours

Limites des SGBD-R

- le modèle de données est trop simple et ne permet pas de représenter facilement les entités du monde réel qui sont souvent plus complexes qu'une relation.
- le modèle relationnel ne permet pas de représenter explicitement les différents types de liens sémantiques qui peuvent lier des entités : composition, généralisation / spécialisation, association...

Limites des SGBD-R

- l'incompatibilité des LMD relationnels et des langages de programmation:
 - les LMD sont déclaratifs et fournissent en résultat un ensemble de tuples, alors que les langages de programmation sont impératifs et travaillent sur un élément à la fois
 - les types de données manipulés par les langages de programmation sont plus complets et plus complexes que ceux des LMD relationnels.

BDO

- Les bases de données orientées objets (BDO) sont nées de la convergence de deux domaines:
 - les bases de données
 - les langages de programmation orientés objets, tels que Eiffel, Smalltalk, C ++ , Java...
 - les objets, qui comportent deux parties: leur valeur, et les opérations, appelées méthodes, qui permettent de les manipuler.
 - L'héritage

BDO

- Les BDO sont caractérisées par 4 points essentiels:
 - un modèle de données qui permet de représenter des structures de données complexes
 - les données et les traitements ne sont plus séparés. La dynamique (les méthodes) fait partie de la déclaration des objets
 - l'héritage
 - tout objet possède une identité qui le distingue de tout autre objet, même s'ils ont la même valeur

Structure de données

- Il existe deux types de liens entre les objets:
 - les liens de composition: "tel objet est composé de tel(s) objet(s)".
 - les liens de généralisation / spécialisation

Structure complexe

- Exemple:
 - CLASS Etudiant1
 - { num : INT ;
 - nom : STRING ;
 - prénoms : LIST STRING ;
 - date-nais : DATE ;
 - adresse : STRUCT {v° : INT ;
 - rue : STRING ;
 - ville : STRING ;
 - pays : STRING } ;
 - cours-suivis : SET STRUCT { nom-cours : STRING;
 - note : FLOAT} }

Lien de composition

CLASS Etudiant2

{ num : INT ;

nom : STRING ;

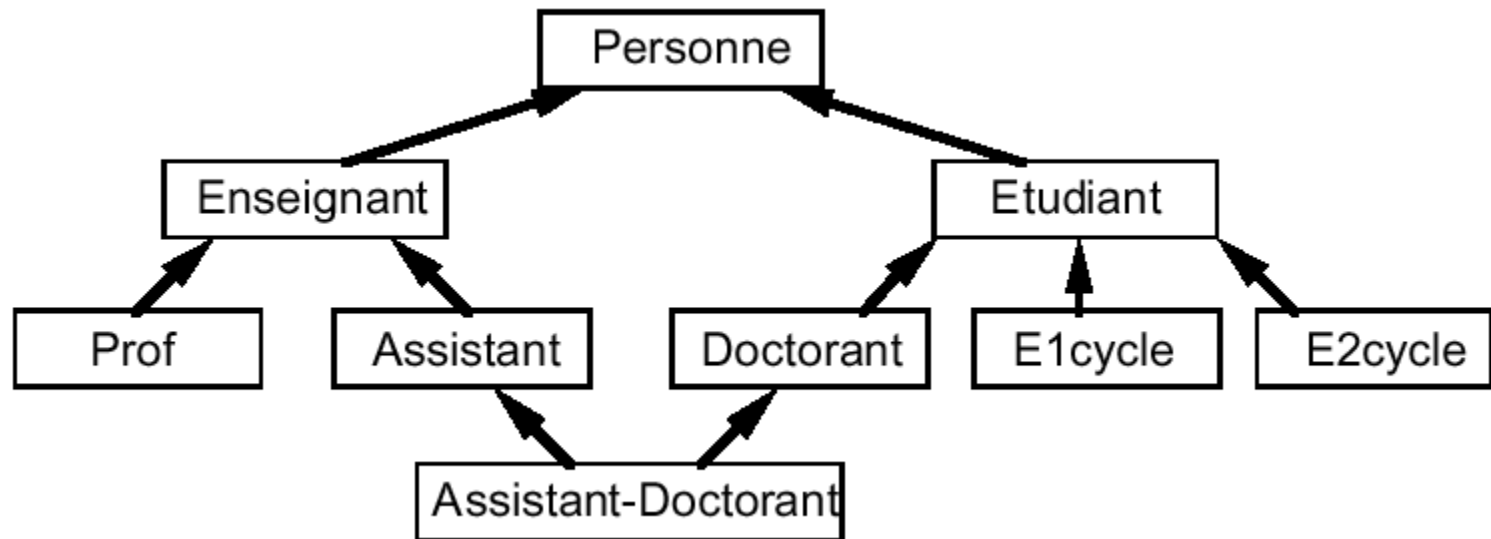
prénoms : LIST STRING ;

cours-suivis : SET STRUCT {**cours** : **Cours** ;
note : FLOAT } }

CLASS Cours

{nomC : STRING ; }

Graphe de généralisation/spécialisation des classes



Code

Class Etudiant

{; }

Class Promo

{ étudiants: **Set** Etudiant ; }

Program

Declare promo95 : Promo;

Declare lucie : Etudiant;

.....

promo95 = Promo ; /* création d'un objet de type Promo */

.....

lucie = Etudiant("Dupont"; ["Lucie"]; 15344; "chemin des oiseaux 2; Lausanne";
1/1/80);

/* création d'un objet Etudiant, dont l'oid est rendu en réponse */

promo95.étudiants.**insert_element**(lucie); /* insertion de l'objet lucie dans l'ensemble
des

étudiants de promo95 /*

Encapsulation

- L'accès aux données des objets se fait à travers des méthodes: on dit que les données sont "encapsulées".
- Seules les méthodes de la classe et de ses sous-classes peuvent accéder directement à la valeur des objets.

Encapsulation

- Avantages:
 - simplifier le travail des programmeurs d'application en ne leur demandant que de connaître le minimum
 - faciliter la maintenance des applications: on peut facilement changer la structure d'un objet...
 - réutiliser au maximum le code des programmes: les méthodes sont écrites une fois pour toutes et utilisées par tous les programmes

Conclusion

- Avantages des BDO
 - ceux de l'approche orientée objets: réutilisation, maintenance facilitée...
 - modèle de données riche
 - prise en compte de la dynamique
 - beaucoup moins d'incompatibilité entre le LMD et le langage de programmation.

Conclusion

- Problèmes:
 - La simplicité du modèle relationnel est perdue.
 - Pas de standard
 - pas encore de
 - méthode complète de conception des données et des méthodes associées.
 - pas encore de méthode complète de conception des données et des méthodes associées.
 - Des travaux sont en cours sur les vues, les versions, l'évolution du schéma, les aspects temporels,...

Plan

- Interface entre un L3G et SGBD (java et Oracle)
- SGBD sur le marché
- DataWarehouse
- Base de données Objet
- Fin cours

FIN du COURS: